



本科生实验报告

实验课程: 操作系统实验

实验名称: 实验 3

专业名称: 计算机科学与技术

学生姓名: 陈安康

学生学号: 22320021

实验地点: 实验中心 D402

实验成绩: _____

报告时间: 2024/3/30

1. 实验要求

通过 MBR 将启动加载器从外存加载到内存，并通过启动加载器完成实地址模式到保护模式的切换。

2. 实验过程

任务 1:

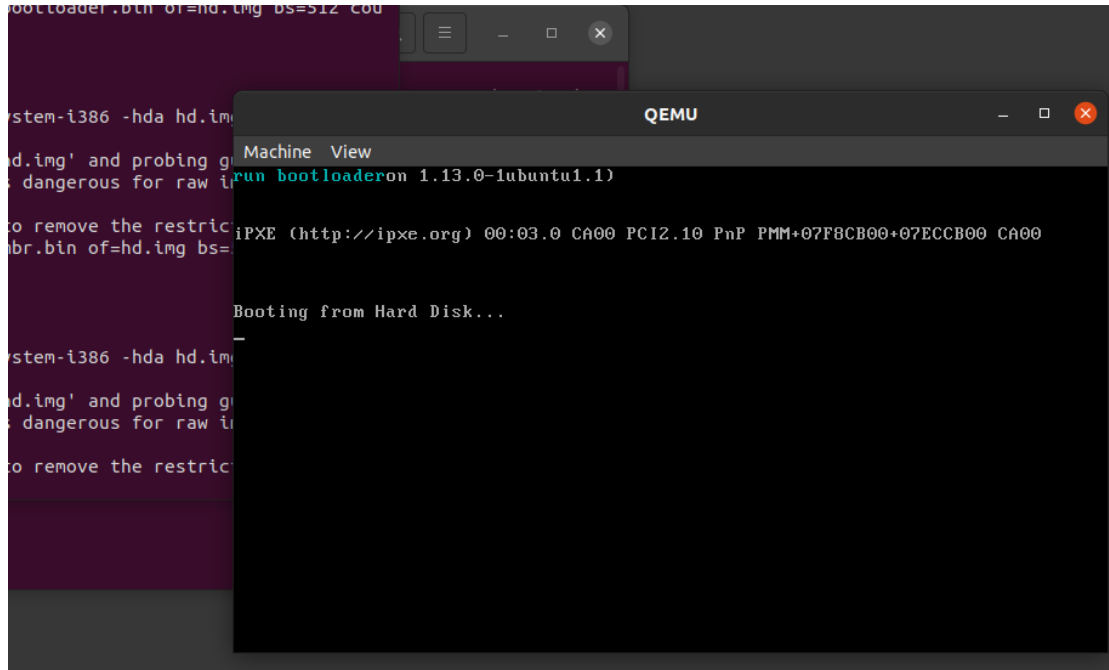
1. 1:

首先编写 bootloader 打印指定字符到显存的汇编代码，这里与 lab2 的打印 hello 到显存的代码思路基本一致，只不过代码起始地址变为 0x7e00。

接着编写 mbr 读取磁盘中的 bootloader 到内存的汇编代码，思路是采用 LBA 方式来读取硬盘，首先将 28 位起始逻辑扇区号和要读取的扇区数量放入对应的端口号，然后请求读取硬盘，检查硬盘状态，直到硬盘准备就绪进行数据读取。

与所给示例代码不同，这里将不再采用循环的形式一个一个扇区读取，而是一次性将五个扇区全部读取到内存中。

结果如下：



代码位于 src 的 mbr.asm 与 bootloader.asm 中。

1.2:

采用 CHS 模式来读取硬盘，经过查找资料，得到 LBA 逻辑扇区号与 CHS 对应的转换公式如下：

$$\text{LBA} = (\text{C} * \text{HPC} + \text{H}) * \text{SPT} + (\text{S} - 1)$$

即：

$$\text{C} = \text{LBA} / (\text{HPC} * \text{SPT})$$

$$\text{H} = (\text{LBA} / \text{SPT}) \% \text{HPC}$$

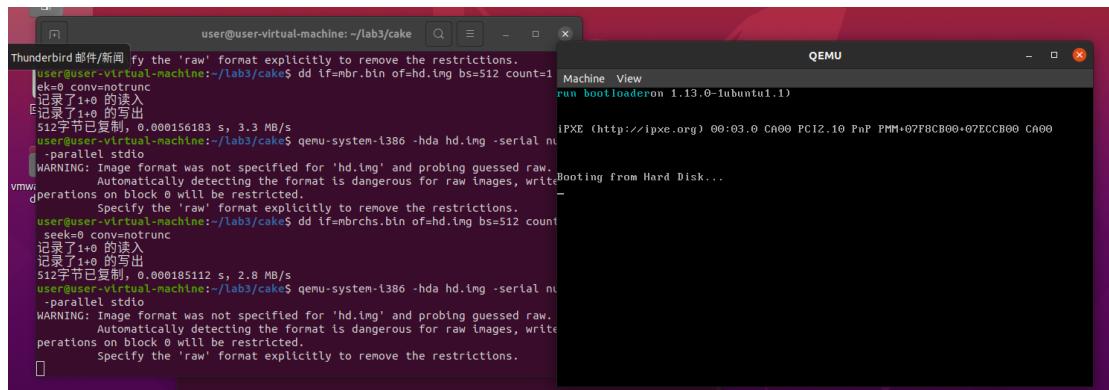
$$\text{S} = (\text{LBA} \% \text{SPT}) + 1$$

其中 LBA 对应 LBA 模式下的扇区号，C 为柱面号，H 为磁头号，S 为扇区号，HPC 为每个柱面的磁头数，在这里是 18，SPT 为每个磁头的扇区数，这里是 63。

实现思路，采用 13H 中断实现，通过功能 02H（读磁盘）来完成磁盘的读取，因为只读入五个扇区且从第 1 个扇区开

始读入，按照上面的转换公式，则 S 为 2，其余的 C，H 均为零，读入内存的扇区个数为 5，再按照规定将对应的数值加载到对应的寄存器中，执行中断即可完成。

结果如下：



```
user@user-virtual-machine: ~/lab3/cake
user@user-virtual-machine:~/lab3/cake$ dd if=mbr.bin of=hd.img bs=512 count=1
1+0 records in
1+0 records out
512 bytes transferred, 0.000156183 s, 3.3 MB/s
user@user-virtual-machine:~/lab3/cake$ qemu-system-i386 -hda hd.img -serial null -parallel stdio
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
user@user-virtual-machine:~/lab3/cake$ dd if=mbrchs.bin of=hd.img bs=512 count=1
1+0 records in
1+0 records out
512 bytes transferred, 0.000185112 s, 2.8 MB/s
user@user-virtual-machine:~/lab3/cake$ qemu-system-i386 -hda hd.img -serial null -parallel stdio
WARNING: Image format was not specified for 'hd.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
```

代码位于 src 中的 mbrchs.asm 中

任务 2:

按照教程，首先编写 MBR 的实现，参考示例代码同样将一次读取一个扇区改为一次性读取五个扇区。

接着完成 bootloader 实现。这里基本参考示例代码，设置好 GDT、打开第 21 根地址线、开启 CRO 保护模式位、远跳转进入保护模式设置好段寄存器并执行打印代码。

设置调试信息，按照教程指引编写好 makefile 和 gdbinit 文件，这里提出一点小疑问，即教程中 gdbinit 里面的加载 bootloader 符号表命令，原教程命令为：“add-symbol-file bootloader.symbol 0x7c00”但这里我感觉应该是笔误设置的加载符号表地址有问题，应该是 0x7e00。这样在 layout.src 时在 0x7e00 处中断才会显示原码。

Make build 编译完成后执行 make debug 进行调试，逐条执行汇编语句，在四个关键地方的显示相关数值如下图：

```

27      ; %eax = 0x80000000
28      mov dword [0x8800+0x18],0x80007fff ; ^^^ ^ ^ ^
29      mov dword [0x8800+0x1c],0x0040920b ; ^' ^ ^ $ ^ ^ ^
30
31      ; ^^^ ^ ^ $ ^ ^ & ^ ^ ^ $ ^ ^ ^ & ^ ^ ^ $ ^ ' ^
32      mov dword [0x8800+0x20],0x0000ffff ; ^^^ ^ ^ ^ ^
33      mov dword [0x8800+0x24],0x00cf9800 ; ^' ^ ^ ^ ^ kb
34
LibreOffice Writer ; ^^^ ^ ^ ^ & ^ ( ^ ^ ^ ' ^ ( ^ ^ ^ ^ ^ ^ DTR
>36      mov word [pgdt], 39 ; ^& ^ ( ^ ^ ' ^ ( ^ ' ^ ' ^ ) ^
37      lgdt [pgdt]
38
39      in al,0x92 ; ^^^ & ^ ( ^ ' ^ ^ ^
40      or al,00000010B
41      out 0x92,al ; ^& ^ ^ ^ A20
42
43      cli ; ^$ ^ & ^ & ^ ^ ^ ^ ^
44      mov eax,cr0

```

```

remote Thread 1.1 In: output bootloader tag
0x8800: 0x00000000      0x00000000      0x00000000      0x00000000
0x8810: 0x00000000      0x00000000
(gdb) ni
(gdb) x/6xw 0x8800
0x8800: 0x00000000      0x00000000      0x0000ffff      0x00000000
0x8810: 0x00000000      0x00000000
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) x/12xw 0x8800
0x8800: 0x00000000      0x00000000      0x0000ffff      0x00cf9200
0x8810: 0x00000000      0x00409600      0x80007fff      0x0040920b
0x8820: 0x0000ffff      0x00cf9800      0x00000000      0x00000000

```

上图可以看到在对 GDT 规定的内存地址赋值之后，内存中的值与我们预期的一致。

```
35      ;^%^ %^ %^ &^ (^ ^ ' (^ %^ %^ %^ DTR
36      mov word [pgdt], 39      ;^&^ (^ ^ ' (^ ' ^ ' ^
>37      lgdt [pgdt]
38
39      in al,0x92                ;^%^ &^ (^ ' ^
40      or al,00000010B
41      out 0x92,al              ;^&^ %^ A20
42
43      cli                      ;^$^ &^ &^ %^
44      mov eax,cr0

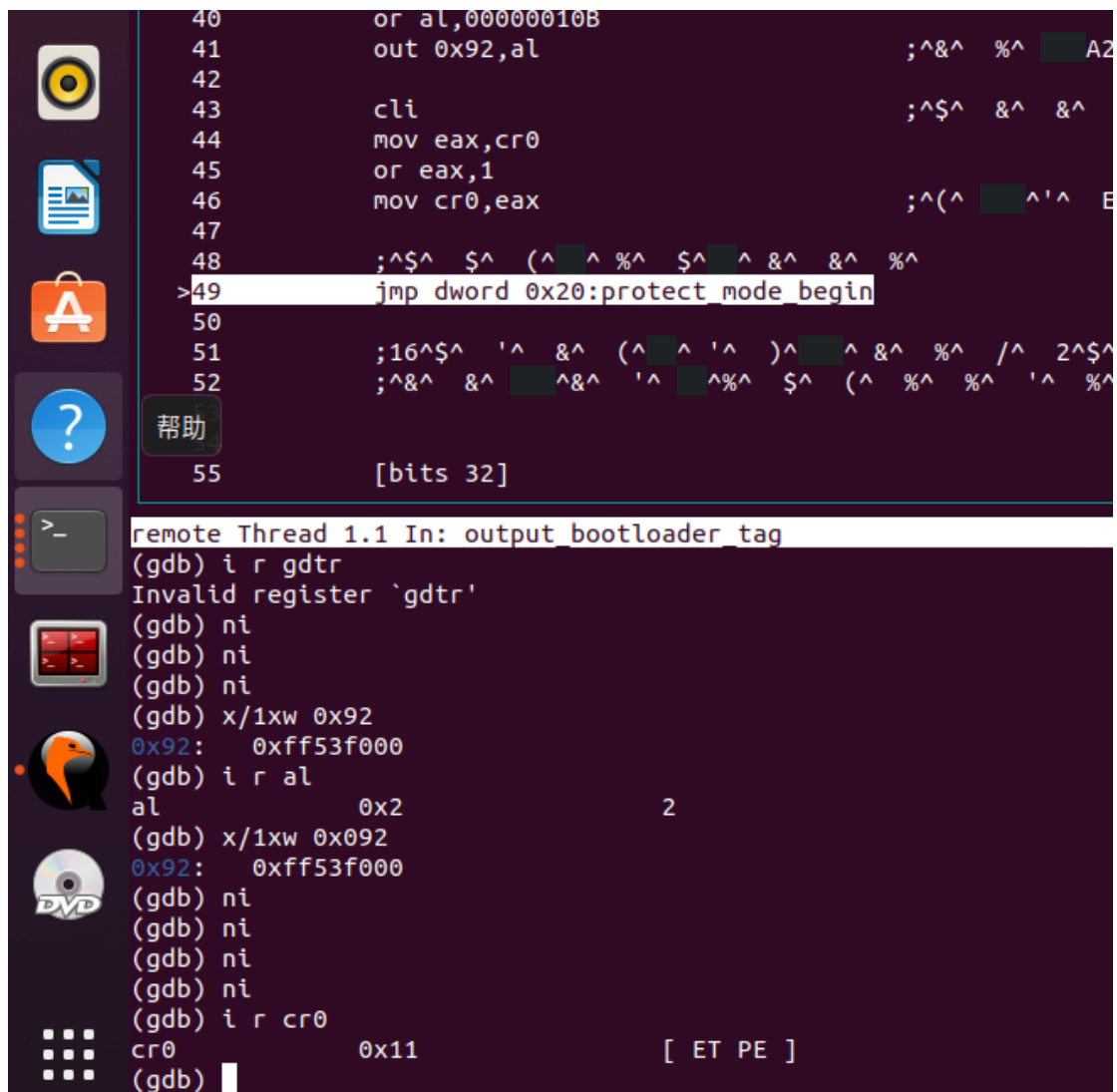
remote Thread 1.1 In: output bootloader_tag
0x8810: 0x00000000      0x00000000
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) x/12xw 0x8800
0x8800: 0x00000000      0x00000000      0x0000ffff      0x00cf9200
0x8810: 0x00000000      0x00409600      0x80007fff      0x0040920b
0x8820: 0x0000ffff      0x00cf9800      0x00000000      0x00000000
(gdb) ni
(gdb) print pgdt
$1 = {<text variable, no debug info>} 0x7ebb <pgdt>
(gdb) x/2xw 0x7ebb
0x7ebb <pgdt>: 0x88000027      0x75720000
(gdb) 
```

不知为什么，我的 gdb 不能直接访问 gdtr 寄存器，所以这里列出 pgdt 的值，可以看到与预期一致。

```
36      mov word [pgdt], 39
37      lgdt [pgdt]
38
39      in al,0x92
40      or al,00000010B
41      out 0x92,al
42
>43      cli
44      mov eax,cr0
45      or eax,1
46      mov cr0,eax
47
48      ;^$^ $^ (^ ^ %^ $^ ^
49      jmp dword 0x20:protect_mo
Ubuntu Software
51      ;16^$^ ' ^ &^ (^ ^ ' ^
52      ;^&^ &^ ^&^ ' ^ ^%
53
54
55      [bits 32]

remote Thread 1.1 In: output bootloader t
0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x0}, ui
xmm6      {v4_float = {0x0, 0x0, 0x0
0x0, 0x0, 0x0}, v2_int64 = {0x0, 0x0}, ui
xmm7      {v4_float = {0x0, 0x0, 0x0
, 0x0}, v4_int32 = {0x0, 0x0, 0x0, 0x1f80
mxcsr      0x1f80      [ IM D
(gdb) i r gdtr
Invalid register `gdtr'
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) x/1xw 0x92
0x92:  0xff53f000
(gdb) i r al
al      0x2      2
```

这一步通过寄存器 a1 作为中间人，先读取端口数值，然后放入 a1 中进行修改，在写入端口中。我无法直接用 gdb 访问 I/O 端口的数据，所以这里最下面列出来了 a1 寄存器的数据，证明这一步这一步操作正确。



The screenshot shows a debugger window with a dark theme. On the left is a sidebar with various icons. The main window is divided into two panes. The top pane displays assembly code with line numbers 40 through 55. Line 49 is highlighted with a yellow background and contains the instruction `jmp dword 0x20:protect_mode_begin`. The bottom pane shows GDB commands and their output. The commands include `(gdb) i r gdtr`, `(gdb) ni`, `(gdb) x/1xw 0x92`, `(gdb) i r al`, `(gdb) x/1xw 0x092`, and `(gdb) i r cr0`. The output for `cr0` is `0x11` with the comment `[ ET PE ]`.

```
40      or al,00000010B
41      out 0x92,al          ;^&^  %^  A2
42
43      cli                  ;^$^  &^  &^
44      mov eax,cr0
45      or eax,1
46      mov cr0,eax         ;^(^  ^'  ^  E
47
48      ;^$^  $^  (^  ^  %^  $^  ^  &^  &^  %^
>49      jmp dword 0x20:protect_mode_begin
50
51      ;16^$^  '  ^  &^  (^  ^  '  ^  )^  ^  &^  %^  /  ^  2^$^
52      ;&^  &^  ^&^  '  ^  ^%  $^  (^  %^  %^  '  ^  %^
53
54      [bits 32]
55
```

remote Thread 1.1 In: output_bootloader_tag
(gdb) i r gdtr
Invalid register `gdtr'
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) x/1xw 0x92
0x92: 0xff53f000
(gdb) i r al
al 0x2 2
(gdb) x/1xw 0x092
0x92: 0xff53f000
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) ni
(gdb) i r cr0
cr0 0x11 [ET PE]
(gdb)

这一步也是一样的思路，通过 `eax` 做中间人改变 `CR0` 寄存器的值，可以看到执行完后 `CR0` 中 `PE` 位置为 1。



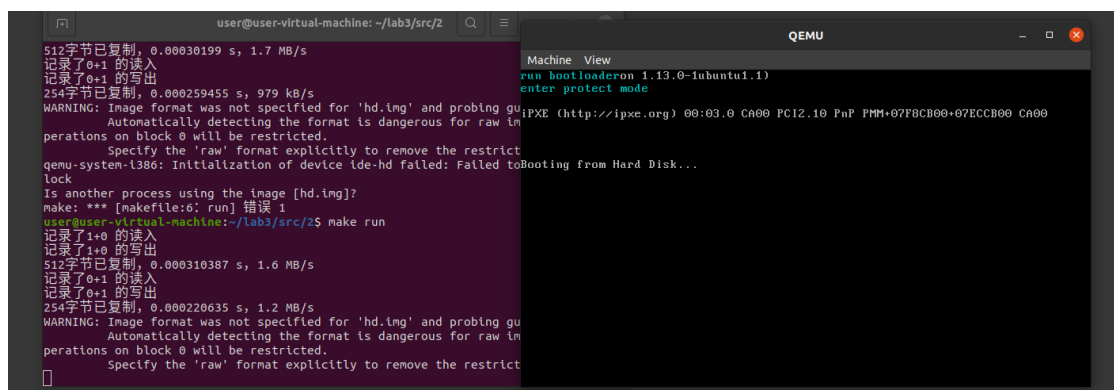
The screenshot shows a debugger window with two panes. The top pane displays assembly code with line numbers 51 through 70. The code includes instructions for setting up segment registers (ds, es, ss, gs) and a jump instruction at line 67. The bottom pane shows a register dump for 'remote Thread 1.1 In: protect mode begin'. The registers listed include eax, ecx, edx, ebx, esp, ebp, esi, edi, eip, eflags, cs, ss, ds, es, fs, gs, and fs_base, each with its current value and offset.

```
51 ;16^$^ ' ^ &^ ( ^ ^ ^ ) ^ ^ &^ % ^ / ^ 2^$^ % ^
52 ;^&^ &^ ^&^ ' ^ ^ % ^ $^ ( ^ % ^ % ^ ' ^ % ^
53
54
55 [bits 32]
56 protect_mode_begin:
57
58 mov eax, 0x8 ;^% ^ ( ^ &^ &^ &^
59 mov ds, eax
60 mov es, eax
61 mov eax, 0x10
62 mov ss, eax
63 mov eax, 0x18
64 mov gs, eax
65
66
67 > jmp $ ; ^&^ % ^ ^ ^
68
69 pgdt dw 0
70 dd 0x8800
```

Register	Value	Offset
eax	0x18	24
ecx	0x0	0
edx	0x80	128
ebx	0x1c	28
esp	0x7c00	0x7c00
ebp	0x0	0x0
esi	0x7ecf	32463
edi	0x0	0
eip	0x7eb9	0x7eb9 <protect_mode_begin+23>
eflags	0x6	[IOPL=0 PF]
cs	0x20	32
ss	0x10	16
ds	0x8	8
es	0x8	8
fs	0x0	0
gs	0x18	24
fs_base	0x0	0

上图可以看到远跳转后 CS 的值已经隐式赋值为 0x20 即代码段的选择子，经过段寄存器赋值，所有的段寄存器的选择子成功被赋值。（因为某些原因，在进行截图的这段代码我把打印程序给删除掉了，不过这并不影响调试观察。）

完整代码执行结果如下：



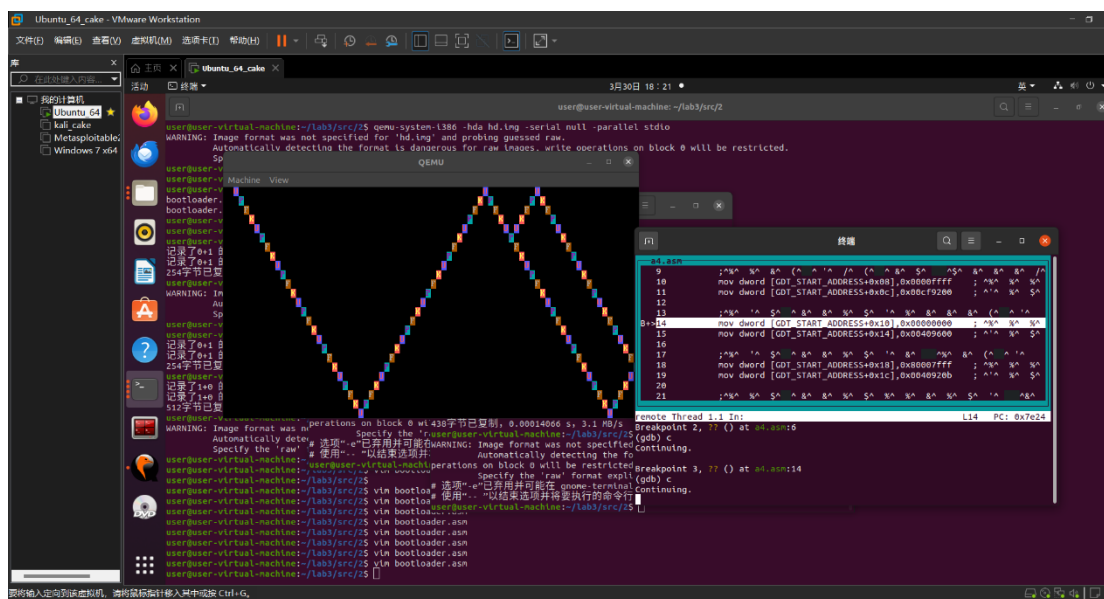
The screenshot shows a terminal window with the output of a bootloader execution. The text includes progress reports for copying files, warnings about image format detection, and the final booting process. A QEMU window is also visible in the background, showing the virtual machine's state.

```
user@user-virtual-machine: ~/lab3/src/2
512字节已复制, 0.00030199 s, 1.7 MB/s
记录了0+1 的读入
记录了0+1 的写出
254字节已复制, 0.000259455 s, 979 kB/s
WARNING: Image format was not specified for 'hd.img' and probing gu
Automatically detecting the format is dangerous for raw im
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrict
qemu-system-i386: Initialization of device ide-hd failed: Failed toBooting from Hard Disk...
lock
Is another process using the image [hd.img]?
make: *** [makefile:0: run] 错误 1
user@user-virtual-machine:~/lab3/src/2$ make run
记录了1+0 的读入
记录了1+0 的写出
512字节已复制, 0.000310387 s, 1.6 MB/s
记录了0+1 的读入
记录了0+1 的写出
254字节已复制, 0.000220635 s, 1.2 MB/s
WARNING: Image format was not specified for 'hd.img' and probing gu
Automatically detecting the format is dangerous for raw im
perations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrict
```

任务 3:

将 lab2 中的 a4.asm 代码稍加修改，移植到 bootloader.asm 中即可完成此目标。这次采用 LCG 算法使得 ax 产生随机字符与颜色。

执行结果如下：



代码位于 src/2 下的 a4.asm 中。

3. 关键代码

任务 1:

以下代码位于 `mbrchs.asm` 中，是采用 13H 中断实现 CHS 模式下磁盘读取的汇编代码（注释是后加的，源代码中没有注释）：

```
mov bx, 0x7e00           ; bootloader 的加载地址
```

```
mov ah, 0x02 ;功能号
```

```
mov al, 5 ;读入硬盘扇区个数
```

```
mov ch, 0 ;柱面
mov cl, 2 ;扇区号
mov dh, 0 ;磁头
mov dl, 0x80 ; 驱动器号
int 0x13
```

任务 2:

在 gdbinit 里面将 bootloader 的符号表加载地址由 0x7c00 改为 0x7e00。

任务 3:

在 lab2 代码基础上加入随机生成字符与颜色的代码:

random:

```
push edx
push ebx
xor eax, eax
xor ebx, ebx
xor edx, edx
mov eax, [randomval]
imul eax, 17
add eax, 12345
mov ebx, 25
idiv ebx
mov eax, edx
```

```
mov bl, 123

imul dx, bx

mov ah, dl

add al, 'A'

mov [randomval], al

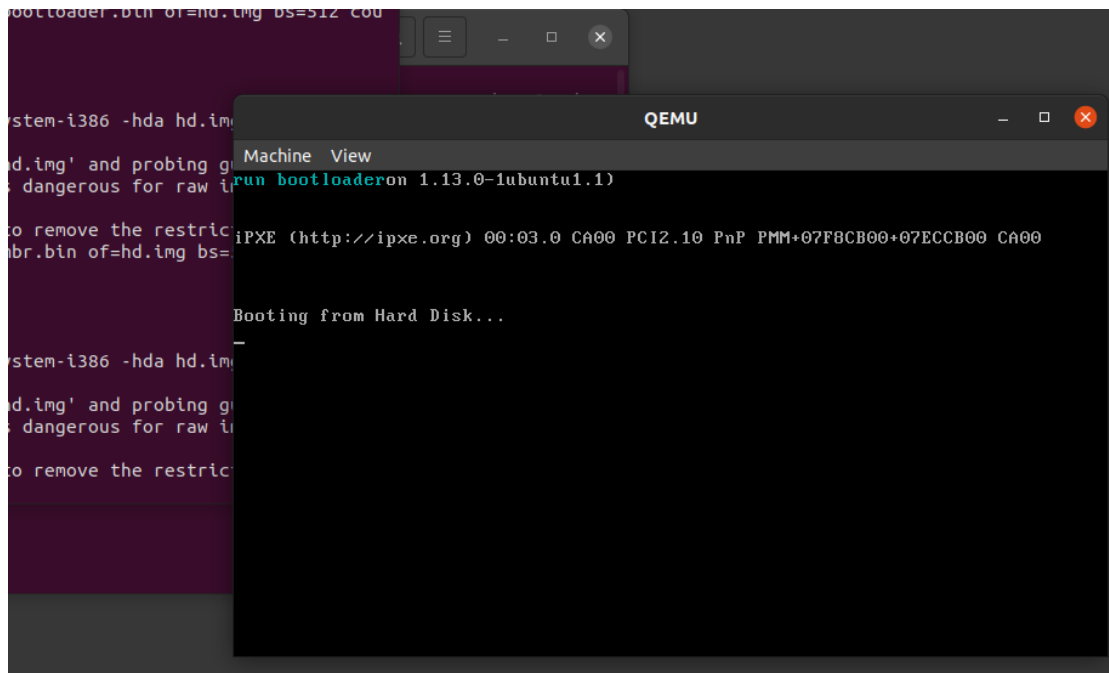
pop ebx

pop edx

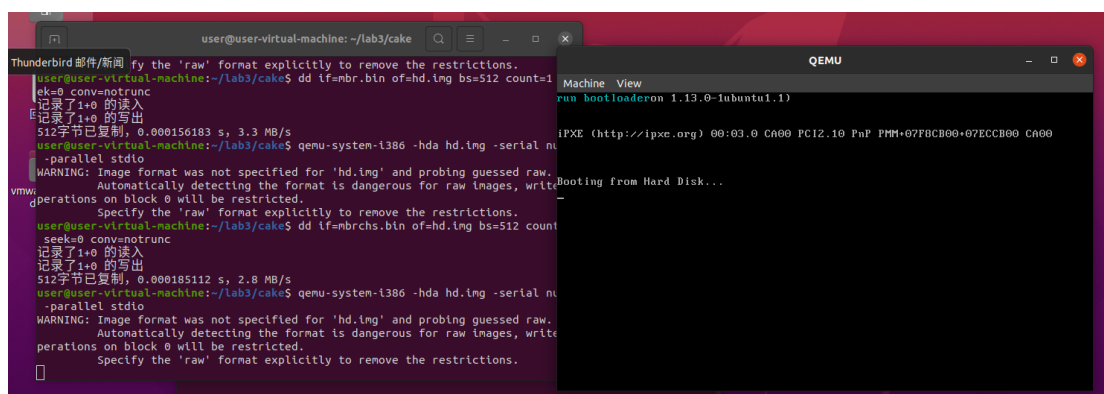
ret

randomval db 0
```

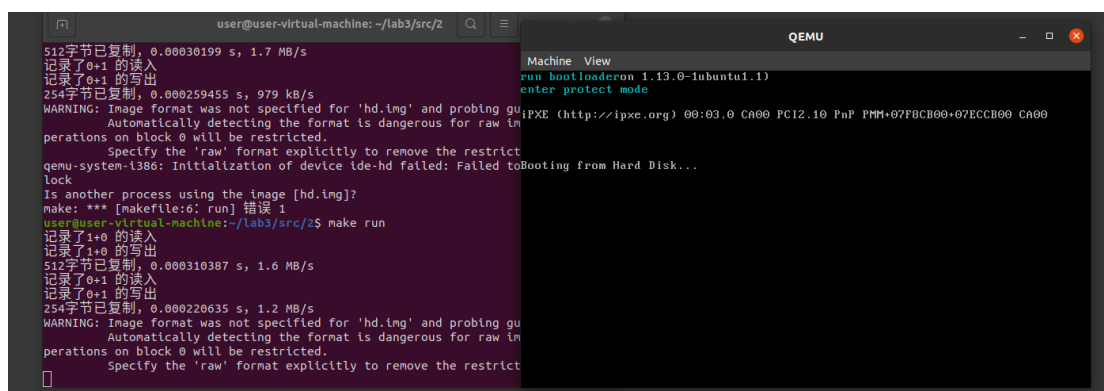
4. 实验结果



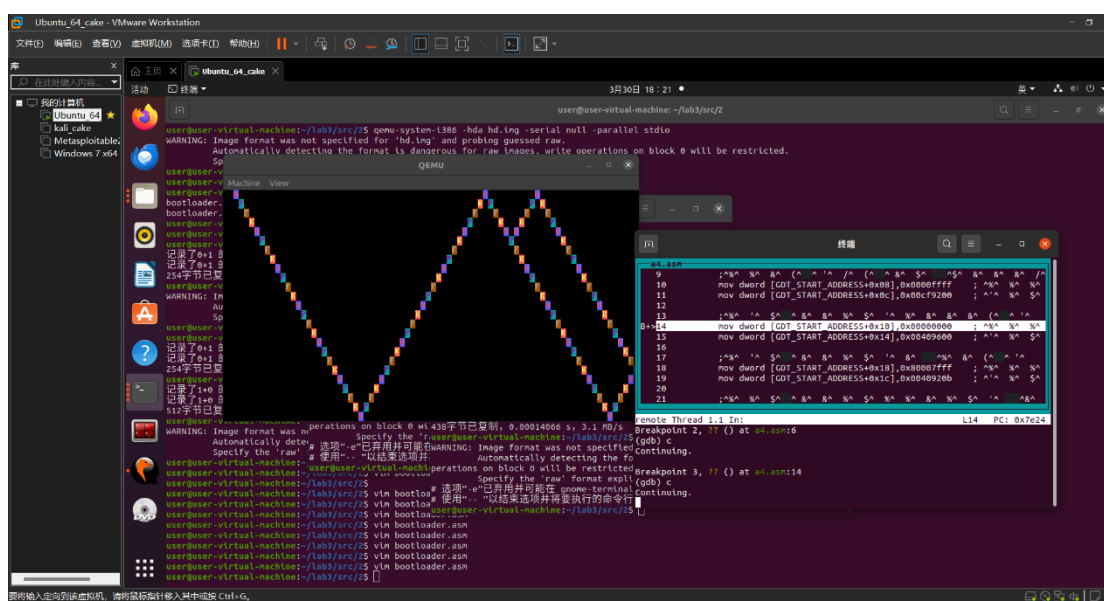
上图为复现 example1 的截图。



上图为采用 CHS 模式读取硬盘完成 bootloader 加载的结果截图。



上图为复现 example2 的结果截图。



上图为任务 3 实验截图。

5. 总结

通过这次学习我了解了操作系统启动加载的流程，明白了从实地址模式到保护模式转换的过程与原理，受益匪浅。